Praveen Madupu

Interview Preparation Series

On-Premises to AWS Migration

SQL Server Focus

Migration Types · Use Cases · Scenario-Based Q&A · FAQs

A complete, interview-ready reference for AWS database & cloud migration roles

Drafted by: Praveen Madupu

Table of Contents

Table of Contents 2
1. How to Use This Guide 3
2. Migration Strategies — The 7 R's 3
   2.1 How to choose 4
3. Types of On-Prem → AWS Migration (by Workload) 4
   3.1 Hosting-model spectrum for SQL Server 4
4. The AWS Migration Toolbox 5
   4.1 Discovery, tracking & server migration 5
   4.2 Database & data-movement tools 5
5. SQL Server on AWS — Deep Dive 6
   5.1 Choosing the target 6
   5.2 Migration methods — Offline vs Online 6
   5.3 Method 1 — RDS native backup & restore from S3 (offline) 7
   5.4 Method 2 — AWS DMS with CDC (minimal downtime) 7
6. End-to-End Use Cases 8
7. Scenario-Based Interview Questions 9
8. Rapid-Fire FAQs 10

1. How to Use This Guide

This guide follows the way a **real AWS migration interview flows** — strategy fundamentals first, then the AWS tooling, then hands-on SQL Server scenarios and rapid-fire FAQs. It covers both infrastructure migration and — in depth — **SQL Server database migration to AWS**.

- **Sections 2–4** build the framework: the 7 R's, workload types, and the AWS migration tool ecosystem.
- **Section 5** is the SQL Server deep dive — RDS vs RDS Custom vs EC2 vs Aurora/Babelfish, plus offline and online methods.
- **Section 6** is end-to-end use cases you can narrate as experience.
- **Sections 7–8** are scenario-based Q&A and FAQs — the parts interviewers drill on.

TIP — Interview framing

For any "how would you migrate X to AWS" question, answer in a fixed order: (1) Assess & discover, (2) pick a strategy and target (RDS / EC2 / Aurora), (3) choose a method (offline vs online / CDC), (4) cut over & validate, (5) optimise & decommission. A repeatable framework beats a tool list.

2. Migration Strategies — The 7 R's

AWS frames migration as the **7 R's**. Name them, define them, and — critically — say ***when*** you'd choose each.

Strategy	What it means	Effort / Risk	Typical AWS target (SQL context)
Retire	Decommission workloads no longer needed.	N/A — removes cost	Nothing — turn it off
Retain (revisit)	Keep on-prem for now (compliance, dependency, latency).	None yet	Hybrid via Direct Connect / VPN
Rehost (Lift & Shift)	Move as-is, no code change.	Low effort, low risk	SQL Server on EC2 (via AWS MGN)
Relocate	Move at hypervisor level, no OS/app change.	Low	VMware Cloud on AWS
Replatform (Lift, tinker & shift)	Small optimisations, no re-architecture.	Low–medium	Amazon RDS for SQL Server
Repurchase (Drop & shop)	Move to a different product / SaaS.	Varies	SaaS; commercial DB → managed offering
Refactor / Re-architect	Redesign, often to cut licensing or gain scale.	Highest effort, highest payoff	Aurora PostgreSQL + Babelfish; serverless

INFO — The two you must not forget

Rehost and Replatform cover the majority of real SQL Server migrations — EC2 for full control, RDS to shed OS/patching/backup toil.

Refactor to Aurora PostgreSQL with Babelfish is the strategic play to escape SQL Server licensing while keeping T-SQL apps working.

2.1 How to choose

- **Data-centre exit / deadline** → Rehost to EC2 with AWS MGN. Fastest path off-prem.
- **Cut OS/patching/backup/HA overhead** → Replatform to Amazon RDS for SQL Server (Multi-AZ handles HA).
- **Need OS access / unsupported feature / specific build** → SQL Server on EC2, or RDS Custom for a middle ground.
- **Escape SQL Server licence cost** → Refactor to Aurora PostgreSQL (SCT + DMS), optionally Babelfish for minimal app change.

3. Types of On-Prem → AWS Migration (by Workload)

A real estate mixes several workload types. Know the AWS tool for each.

Workload type	What moves	Primary AWS tool	Target
Servers / VMs	Physical + VMware/Hyper-V VMs	AWS Application Migration Service (MGN)	Amazon EC2
SQL databases	SQL Server, Oracle, MySQL, PostgreSQL	AWS DMS (+ Schema Conversion / SCT)	RDS, RDS Custom, EC2, Aurora
File / NAS data	File shares, unstructured data	AWS DataSync, Storage Gateway	Amazon S3, EFS, FSx
Bulk / offline data	TB–PB where bandwidth is poor	AWS Snow Family (Snowball)	S3 → then restore
Web / app tier	Web apps, middleware	MGN, then containerise	EC2, ECS/EKS, Elastic Beanstalk
Identity	On-prem Active Directory	AWS Directory Service / AD Connector	AWS Managed Microsoft AD
Discovery / tracking	Inventory + dependencies	Application Discovery Service + Migration Hub	Migration Hub dashboard

3.1 Hosting-model spectrum for SQL Server

- **IaaS — SQL Server on EC2**: full control, you patch the OS and manage HA (Always On AG / FCI). Best for legacy or unsupported features.
- **Managed — Amazon RDS for SQL Server**: AWS handles OS, patching, backups and Multi-AZ HA. Less control, far less toil.
- **Middle ground — RDS Custom for SQL Server**: managed convenience plus OS/DB-level access for apps that need it.
- **Refactor — Aurora PostgreSQL (+ Babelfish)**: cloud-native, licence-free engine; heterogeneous move via SCT + DMS.

4. The AWS Migration Toolbox

AWS Migration Hub is the central place to discover, plan and track migrations across tools and accounts. Around it sit the specialised services below.

4.1 Discovery, tracking & server migration

Service	Purpose	When to use
AWS Migration Hub	Central discovery, planning & progress tracking.	Single pane of glass for the whole program
Application Discovery Service	Agentless (connector) or agent-based inventory + dependency data.	Build the estate map & migration waves
AWS Application Migration Service (MGN)	Primary lift-and-shift service; block-level replication to EC2.	Rehosting VMs/servers to EC2 (replaces SMS/CloudEndure)
AWS Migration Acceleration Program (MAP)	Methodology + funding: Assess → Mobilize → Migrate & Modernize.	Framing a large program

4.2 Database & data-movement tools

Tool	Purpose	When to use
AWS DMS (Database Migration Service)	Full-load + change data capture (CDC) migrations, homogeneous or heterogeneous.	SQL → RDS/EC2/Aurora with minimal downtime
AWS SCT / DMS Schema Conversion	Convert schema & code across engines; assess conversion effort.	SQL Server → Aurora PostgreSQL/MySQL
Native backup / restore to S3	RDS SQL Server restores .bak files from S3 via stored procedures.	Simple offline RDS migration
AWS DataSync	Fast, managed file/object transfer online.	File shares, backup files to S3
AWS Snow Family (Snowball)	Physical appliance for offline bulk transfer.	TB–PB data / poor bandwidth

TIP — DMS vs SCT — the classic trap

SCT (Schema Conversion Tool / DMS Schema Conversion) converts the schema, stored procedures and code — used for heterogeneous moves (e.g. SQL Server → Aurora PostgreSQL).

DMS moves the data (full load) and keeps the target in sync (CDC). For homogeneous SQL→SQL you often need only DMS; for SQL→Aurora you use SCT first, then DMS.

5. SQL Server on AWS — Deep Dive

Two decisions drive the whole migration: (1) which target, and (2) offline vs online method.

5.1 Choosing the target

	RDS for SQL Server	RDS Custom	SQL Server on EC2	Aurora PostgreSQL + Babelfish
Model	Managed (PaaS)	Managed + access	Self-managed (IaaS)	Managed, cloud-native
OS access	No	Yes	Yes	N/A
Compatibility	High (some limits)	High	100% SQL Server	T-SQL via Babelfish (not 100%)
HA	Multi-AZ	Multi-AZ	You build (AG/FCI)	Aurora replicas / Multi-AZ
Patch/backup	AWS-managed	Shared	You own	AWS-managed
Best for	Standard SQL apps, less toil	Apps needing OS/agents	Legacy / unsupported features	Escaping SQL licence cost

INFO — Rule of thumb

Want managed with minimal change → RDS for SQL Server. Need OS access or third-party agents → RDS Custom or EC2.

Need 100% feature parity / specific build / clustering control → SQL Server on EC2.

Want to drop SQL Server licensing → refactor to Aurora PostgreSQL, using Babelfish to keep T-SQL apps mostly unchanged.

WATCH OUT — Know RDS for SQL Server limitations

No OS/host access; can't RDP to the box. No FILESTREAM, limited SQL Agent features, and some cross-instance features are restricted.

Storage/size ceilings per instance; certain editions/features (e.g. some Always On configs) aren't available. If a requirement hits these, choose EC2 or RDS Custom.

5.2 Migration methods — Offline vs Online

Offline = app is down for the data copy. **Online** = source stays live, changes stream via CDC, and you cut over with minimal downtime.

Method	Mode	Target	Notes
Native backup/restore via S3	Offline	RDS	rds_backup_database / rds_restore_database procs
Backup/restore (.bak)	Offline	EC2	Full native restore; simplest for IaaS
BACPAC import/export	Offline	RDS / EC2	Schema+data; small–medium DBs
AWS DMS — full load	Offline	RDS/EC2/Aurora	One-time bulk load
AWS DMS — full load + CDC	Online	RDS/EC2/Aurora	Continuous sync, minimal-downtime cutover
Log shipping	Online-ish	EC2	Ship + restore logs, final tail-log cutover
Always On Distributed AG	Online	EC2	Extend AG to AWS, then fail over
Transactional replication	Online	EC2 / RDS	Granular, phased cutover
Snowball + restore	Offline	EC2 / RDS(S3)	Huge backups over poor links

5.3 Method 1 — RDS native backup & restore from S3 (offline)

The simplest managed path: back up on-prem, upload the .bak to S3, and restore into RDS with stored procedures (requires the SQLSERVER_BACKUP_RESTORE option group + an IAM role granting S3 access).

```
-- On the RDS instance (msdb): restore a .bak sitting in S3
EXEC msdb.dbo.rds_restore_database
     @restore_db_name   = 'SalesDB',
     @s3_arn_to_restore_from = 'arn:aws:s3:::champs-migration/SalesDB_full.bak';

-- Track progress of the restore task
EXEC msdb.dbo.rds_task_status @db_name = 'SalesDB';

-- (Reverse direction) native backup FROM RDS to S3
EXEC msdb.dbo.rds_backup_database
     @source_db_name    = 'SalesDB',
     @s3_arn_to_backup_to = 'arn:aws:s3:::champs-migration/SalesDB_out.bak',
     @overwrite_s3_backup_file = 1;
```

WATCH OUT — Offline caveat

Native restore is offline — the app is down from the last on-prem backup until cutover. For large or busy databases use DMS with CDC instead to keep downtime to minutes.

5.4 Method 2 — AWS DMS with CDC (minimal downtime)

The near-zero-downtime narrative to walk an interviewer through:

1. Assess with DMS Schema Conversion / SCT (only heterogeneous needs schema conversion; SQL→SQL is homogeneous).
2. Provision a DMS replication instance in the VPC with line-of-sight (Direct Connect / VPN) to the on-prem SQL Server.
3. Create source and target endpoints; ensure the source is in FULL recovery model so DMS can read the transaction log for CDC.
4. Run a task in 'Full load + ongoing replication (CDC)' mode — bulk load then continuous change capture.
5. Validate row counts, objects, logins and jobs; monitor CDC latency until it's near zero.
6. Cut over in a low-traffic window: stop writes on-prem, let CDC drain, repoint the app connection string.
7. Bake in, then decommission the source.

INFO — What DMS does NOT move

DMS migrates table data (and optionally basic schema) — it does NOT move SQL Agent jobs, logins/users, linked servers, or server-level objects. Script those separately (e.g. with dbatools) and apply them to the target.

Migrate logins and jobs to an EC2 target with dbatools before cutover:

```
$src = 'ONPREM\PROD'
$tgt = 'ec2sql01.internal'   # or RDS endpoint for supported objects

# Logins with SIDs preserved so app auth survives cutover
Copy-DbaLogin      -Source $src -Destination $tgt -ExcludeSystemLogins
Copy-DbaAgentJob   -Source $src -Destination $tgt   # EC2 target
Copy-DbaLinkedServer -Source $src -Destination $tgt
```

6. End-to-End Use Cases

Concrete, narratable scenarios — the "tell me about a migration you did" answers that land well.

Use Case 1 — Data-centre exit under a deadline

Situation: Lease ending in 4 months; ~100 VMs and 25 SQL Server databases.

Approach: Rehost VMs with AWS MGN to EC2; lift SQL to SQL Server on EC2 for full compatibility. Modernise to RDS later.

Why: Deadline beats optimisation — move first, refactor in place afterwards.

Use Case 2 — Cut DBA operational overhead

Situation: A small team overloaded with patching, backups and HA for a dozen SQL instances.

Approach: Replatform to Amazon RDS for SQL Server with Multi-AZ; migrate via DMS full-load + CDC for minimal downtime.

Outcome: Automated backups, patching and failover — operational toil drops sharply.

Use Case 3 — Escape SQL Server licensing

Situation: Leadership wants to cut recurring SQL Server licence spend.

Approach: Refactor to Aurora PostgreSQL. Use SCT to convert schema/code, DMS to move data, and Babelfish so existing T-SQL apps keep working with minimal change.

Use Case 4 — Very large database, poor bandwidth

Situation: A 10 TB database plus 60 TB of backups; thin WAN link.

Approach: Seed with AWS Snowball (ship backups offline to S3), restore, then use DMS CDC to catch up the delta before cutover.

Use Case 5 — Hybrid, phased migration

Situation: Compliance requires some data stays on-prem for now.

Approach: Retain sensitive workloads on-prem; connect via Direct Connect/VPN; migrate the rest in waves identified by dependency analysis.

7. Scenario-Based Interview Questions

These test judgement. Answer with the framework: assess → target → method → cut over → validate.

S1 Migrate a 4 TB SQL Server database to AWS with under 15 minutes of downtime. How?

A: Use an online method so the source stays live during the bulk copy. Run AWS DMS in 'full load + CDC' mode: DMS bulk-loads the data, then continuously captures changes from the transaction log. Cut over by stopping writes, letting CDC latency drain to zero, and repointing the app.

A: For very large seeds you can pre-seed with a native backup (S3/Snowball) and let DMS CDC handle only the delta. Native restore alone is offline and won't hit a 15-minute window at 4 TB.

- Ensure the source is in FULL recovery model so DMS can read the log for CDC.

S2 The app needs OS access, a third-party backup agent, and FILESTREAM. Which AWS target?

A: SQL Server on Amazon EC2. RDS for SQL Server blocks OS access and doesn't support FILESTREAM or arbitrary agents. RDS Custom gives OS access but EC2 is the safest choice when full parity and third-party agents are required.

S3 Leadership wants to eliminate SQL Server licence costs. What's your migration path?

A: Refactor to Aurora PostgreSQL. Use SCT (DMS Schema Conversion) to convert schema and stored procedures and to quantify conversion effort, then AWS DMS to migrate data with CDC. Add Babelfish for Aurora PostgreSQL so existing T-SQL/TDS applications connect with minimal code change, reducing rewrite risk.

S4 How do you migrate a database to RDS for SQL Server with the least tooling?

A: Native backup and restore via S3. Add the SQLSERVER_BACKUP_RESTORE option group and an IAM role for S3, upload the .bak to S3, then run msdb.dbo.rds_restore_database and poll rds_task_status. It's offline, so schedule it in a maintenance window; for large/busy DBs prefer DMS with CDC.

S5 You have 60 TB of backups to move and a slow WAN link. What do you use?

A: AWS Snow Family (Snowball / Snowball Edge). Copy the backups to the appliance on-site and ship it; AWS loads the data into S3. Then restore in AWS and use DMS CDC to reconcile any changes since the backup.

S6 After cutover to a new SQL instance on AWS, the app can't authenticate. Likely cause?

A: Orphaned users — logins weren't migrated, or their SIDs don't match the database users (DMS moves data, not logins). Recreate logins with the original SIDs, then fix mappings. Also check security groups, connection string/endpoint, and TLS settings.

```
-- Detect orphaned users in the migrated database
SELECT dp.name AS orphaned_user, dp.type_desc
FROM sys.database_principals dp
LEFT JOIN sys.server_principals sp ON dp.sid = sp.sid
WHERE sp.sid IS NULL
  AND dp.type IN ('S','U','G')
  AND dp.name NOT IN ('dbo','guest','sys','INFORMATION_SCHEMA');
```

S7 DMS CDC latency keeps climbing during migration. How do you troubleshoot?

A: Check the replication instance size (CPU/memory/network) and scale it up if saturated; verify the source transaction log isn't being truncated before DMS reads it (keep FULL recovery + frequent log backups); look for large transactions or LOB columns slowing CDC; reduce concurrent load; and confirm target write throughput (IOPS/instance class) isn't the bottleneck. CloudWatch DMS metrics (CDCLatencySource/Target) pinpoint which side is behind.

S8 How do you validate the migration before decommissioning the source?

A: Compare object and row counts on critical tables (DMS data validation can automate this), verify logins/users/permissions, confirm SQL Agent jobs and linked servers exist on the target, run DBCC CHECKDB, execute application smoke tests, and compare performance. Keep the source available read-only for a bake-in period as a rollback path.

S9 Costs are higher than forecast after migration. How do you optimise?

A: Right-size instances from real utilisation (AWS Compute Optimizer), buy Reserved Instances or Savings Plans for steady workloads, use BYOL / License Mobility for SQL where cheaper than License-Included, move cold backups to S3 lifecycle tiers, and remove orphaned EBS volumes/EIPs. Cost Explorer and Trusted Advisor surface the wins.

8. Rapid-Fire FAQs

Q. What are the 7 R's of AWS migration?

Retire, Retain, Rehost, Relocate, Replatform, Repurchase, and Refactor/Re-architect. They range from doing nothing (retain/retire) through lift-and-shift (rehost/relocate) to full redesign (refactor).

Q. What is AWS Application Migration Service (MGN)?

The primary lift-and-shift (rehost) service. It uses block-level replication to continuously copy source servers into AWS, then launches them as EC2 instances with minimal cutover downtime. It replaced the older Server Migration Service and CloudEndure Migration.

Q. DMS vs SCT — what's the difference?

AWS DMS moves the data (full load) and keeps it in sync (CDC). SCT (now DMS Schema Conversion) converts the schema, code and stored procedures between different engines. Homogeneous SQL→SQL usually needs only DMS; heterogeneous SQL→Aurora needs SCT first, then DMS.

Q. What is Babelfish for Aurora PostgreSQL?

A capability that lets Aurora PostgreSQL understand the SQL Server wire protocol (TDS) and T-SQL, so applications written for SQL Server can connect to Aurora with little or no code change — the key enabler for refactoring off SQL Server licensing.

Q. When would you choose EC2 over RDS for SQL Server?

When you need OS-level access, third-party agents, FILESTREAM, specific SQL builds/editions, custom clustering (Always On AG/FCI), or 100% feature parity — and you accept managing patching, backups and HA yourself. RDS Custom is a middle ground when you need OS access but still want managed convenience.

Q. What's the difference between offline and online migration?

Offline takes the source down for the whole copy (e.g. native restore). Online keeps the source live and streams changes via CDC (e.g. DMS full-load + CDC), limiting downtime to a short final cutover.

Q. How do you restore a database into RDS for SQL Server?

Add the SQLSERVER_BACKUP_RESTORE option group and an IAM role with S3 access, upload the .bak to S3, then run msdb.dbo.rds_restore_database and monitor with rds_task_status. RDS also supports native backup to S3 via rds_backup_database.

Q. What connectivity options link on-prem to AWS during migration?

AWS Site-to-Site VPN for quick/temporary encrypted links, and AWS Direct Connect for dedicated, private, high-bandwidth connectivity. Both give DMS and MGN line-of-sight to source systems.

Q. What is the AWS Migration Acceleration Program (MAP)?

AWS's structured methodology (and funding) for large migrations, organised into three phases: Assess (readiness & business case), Mobilize (build the foundation, landing zone, skills), and Migrate & Modernize (execute at scale).

Q. What is a landing zone in AWS?

A pre-configured, secure, multi-account AWS environment (networking, identity, guardrails, logging) — often built with AWS Control Tower — so migrated workloads land in a governed, compliant setup from day one.

Q. What does AWS DMS NOT migrate?

DMS moves table data and can create basic schema, but it does not move SQL Agent jobs, logins/users, linked servers, or server-level configuration. Migrate those separately (e.g. with dbatools) and preserve login SIDs to avoid orphaned users.

Q. How do you migrate 100 TB with limited bandwidth?

Use the AWS Snow Family (Snowball / Snowball Edge) for offline bulk transfer into S3, then restore in AWS and reconcile the delta with DMS CDC. This avoids saturating the WAN link.

Q. What is a migration wave?

A batch of interdependent servers/apps (identified via Application Discovery Service dependency mapping) migrated together, so tightly coupled workloads move in one cutover rather than being split.

Q. How do you reduce SQL Server cost on AWS?

Right-size with Compute Optimizer, use Reserved Instances/Savings Plans, choose BYOL/License Mobility where cheaper than License-Included, tier backups in S3, and clean up orphaned EBS/EIP resources. Cost Explorer and Trusted Advisor guide the choices.

Prepared for interview preparation
sqldbachamps.com